

拓扑排序应用软件个人任务及感想

一 我承担的任务

我在项目中担任组员 B，主要负责 GUI 主框架与交互控制模块：从最开始的主窗口搭建，到 MainController 的任务调度，再到 ResultPanel 的分页结果列表、UIStyle 的统一界面样式，最后把 A、C、D 三个模块的代码整合到一个能跑的完整程序里。除了开发工作，我还承担了项目统筹的角色，负责搭 GitHub 仓库、拉取合并各分支的代码、组织了五次线上会议、对齐每个人的接口和进度，最后整理项目目录和 README 文档。

在模块对接过程中，我和 A 核对算法返回结果里哪些字段表示结果是否完整、停止原因怎么转成中文提示；和 C 检查点击结果列表时图上的颜色高亮和节点序号对不对得上；和 D 确认导出文件的时候，怎么把"有没有算完、为什么停下来"这些信息传给导出模块，不能把截断的结果标成全部；也给 E 提供了演示流程和用户手册需要的界面说明素材。我的工作其实就是做中间层，把各写各的模块串成用户能直接用的软件。

二 对工程实践的体会

以前做课程作业的时候，程序只要能跑出正确答案就算完成了，从来没考虑过界面会不会卡、用户能不能取消、报错了用户看不看得懂。真正做多模块项目才发现，这些问题比算法本身还费时间。我最开始以为写 Swing 界面就是拖一拖组件，拼个窗口就完事，真做起来才知道，后台线程调度、事件回调不阻塞 UI、列表分页渲染这些细节，才是决定软件能不能用的关键。比如最开始计算的时候直接在主线程跑，点完计算整个界面就卡死了，用户以为程序崩了，改成 SwingWorker 后台跑才解决。

跨模块对接让我对接口设计有了完全不一样的理解。最开始大家各写各的，对同一个数据结构的理解都不一样：A 的枚举结果里有个 isComplete 字段，他以为我知道这表示"是不是把所有可能的序列都找完了"，我一开始以为

这只是"列表有没有填满",两边理解差了十万八千里,联调的时候来回改了好多次。我作为中间层,把 A 的算法结果、C 的画布、D 的导出串起来的过程中才真正体会到,好的接口不是方法名和参数对上就行,还要让两边对每个字段是什么意思理解得一模一样,不然到了联调全是坑。

做测试和修 bug 的过程也改变了我对"做完了"的判断。比如导出这个功能,最开始我觉得把结果写到 TXT 文件里就算完了,后来 D 提出来,任务书要求导出文件必须写明结果是不是完整的、有没有到 1000 条上限、是用户自己取消的还是算完了。这些要求不是写代码的时候拍脑袋能想到的,都是在反复对接、测试、改需求的过程中一点点补全的。还有一次发现点"取消计算"根本点不动,查了半天才知道是因为图太小、算法跑太快,一秒就算完了,根本来不及点,不是取消功能坏了。这种问题不实际测根本发现不了。

三 团队合作中的体会

和组员一起做项目,我最大的感受是版本管理真的太重要了。最开始大家各推各的,有人在群里说"我改了接口",另一个人没看见,第二天拉代码下来直接编译不过,浪费了大半天时间。后来我们慢慢定了规矩:每次开会第一件事先拉最新代码,谁改了接口必须在群里 @ 所有人,新功能写完了在群里说一声,才慢慢顺起来。之前我总觉得 Git 就是个存代码的地方,现在才知道它是多人协作的骨架,没有分支和合并,五个人根本没法一起写一个项目。

和 C 对接可视化效果的时候,我真切体会到站在用户角度看问题有多重要。我在自己这边测,觉得图上节点颜色变了、能点结果就挺好了,C 说你点第 2 条结果看看,我才发现所有结果都是一个颜色,根本区分不开;又试了放大查看窗口,点完切换成环形布局,弹出的新窗口居然又变回分层了。这些问题我自己测的时候根本想不到去试,真的拿起来用一遍全出来了。后来我们每次改完交互,都故意乱点一通、来回切,就怕漏了什么边界情况。

作为负责统筹的人,我学会了怎么把一个大项目拆成每个人能独立干的小活。最开始第一次开会,大家坐在一起你看我我看你,都不知道从哪下手。我们花了一晚上把任务拆成 A 到 E 五个模块,每个人写哪几个类、和哪个人对哪

个接口，写得明明白白，散会之后大家就知道回去该干什么了。中间遇到接口冲突、合并代码打架的时候，我负责把两边拉到一起对齐，虽然都是碎活，但是能保证项目不会因为谁等谁而停在那。

Git 分支开发的方式也让我明白为什么公司里不允许所有人直接往主分支上提交。每个人在自己的分支上写，写得差不多了再合并，出了问题也不会影响主分支。我们的 `dev-b` 分支就是集成分支，把大家的代码拼到一起测，测没问题了再合到 `main`，就是把"开发"和"稳定"分开，不会出现主分支一直是坏的、谁拉下来都跑不起来的情况。

四 不足与改进

回头看整个过程，我也有不少做得不好的地方。最开始做界面交互的时候，我没把所有操作方式列出来想清楚，结果"按住节点拖动"和"单击节点高亮"是同一个操作，功能冲突了，到后期改了好半天；还有缩放按钮，一开始放工具栏上大家说不方便，又挪到画布右下角，中间来回调了好多次。以后做界面设计，应该先把所有交互方式列出来，确认不会冲突、位置合理了再动手写，不要写完再返工。

我在文档整理上走了很多弯路。最开始没定好目录结构，文件到处乱放，一会儿放根目录一会儿放 `docs` 下面，后面整理的时候来回挪，浪费了好多时间。还有生成报告的脚本、临时的 `docx` 文件，一开始没加 `gitignore`，不小心提交到仓库里了，又得删掉。如果项目第一天就把目录怎么放、哪些文件不入仓定好，后面根本不用花这么多时间收拾。

联调的时候我大部分还是靠手动跑程序、点界面来测，不少场景要临时把代码拼在一起才能试。其实像"计算中途取消""连续算两次""导出结果完整性"这些高频场景，应该写成固定的测试入口，每次改完代码跑一遍，不用每次都手动点，省下来的时间能多测几个边界情况。

五 小结

这次实践让我从“把界面画出来”这个层面，真正理解了“把几个人写的代码拼成一个用户能用的软件”中间到底要做多少事。我不仅学会了怎么写 Swing 界面、怎么在后台跑任务不卡 UI、怎么把不同模块的接口接起来，更学会了怎么和四个人一起协作：怎么定接口、怎么合并代码、怎么开会对齐进度、怎么把乱掉的文件整理清楚。虽然中间改了无数次 bug、合并了几十次代码，一开始还总出冲突，但是最后看到软件从输入关系、出图、列结果到导出文件全流程跑通的时候，那种成就感真的很强。这门课教的不只是拓扑排序这个算法，更多的是怎么和人一起把算法做成一个真的能用的东西。

这次实践让我从“把算法写出来”进一步认识到“让算法成为可靠软件的一部分”需要哪些工作。我既加深了对图结构、拓扑排序和枚举限制的理解，也学会了用明确的接口、具体的反馈和可复现的验证与组员合作。今后面对类似项目，我会继续重视实现质量，同时更主动地考虑调用者、使用者和协作成员的需要，把这些经验用于后续的学习与开发。